



P0-Finalen 2023

Lösungsförslag

Inflation

Av: Johan Sannemo

Testfallsgrupp 1 (Alla tal är procentenheter)

Testfall 1: 100% + 100% = 200%

Testfall 2: 100% - 100% = 0%

Testfall 3: 100% * 100% = 10000%

Testfall 4: 100% / 100% = 1%

Allmän lösning

• Allmän lösning

Synonymer

Av: Johan Sannemo

Testfallsgrupp 1 (Alla ord på andra språket har två översättningar)

Testfall 1: En ordlista med två ord på svenska och ett ord på engelska. Detta testfall är avsett för att testa om systemet kan hantera ord som har två översättningar.

Allmän lösning

• Allmän lösning

Brobyggge

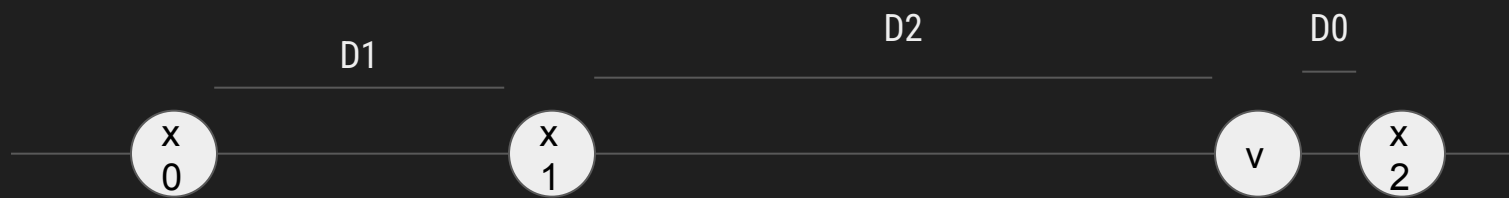
Av: Herman Lauenstein, Joshua Andersson

Vattenfall

Av: Ivar Källström, Joshua Andersson

Sågverket

Av: Nils Gustafsson



Testfallsgrupp 1 (N=1)

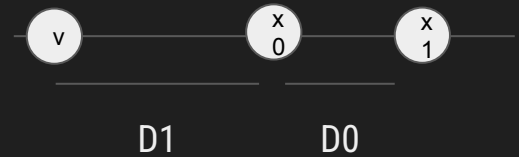
Det finns endast ett x vi måste hitta, x_0

Insikt 1: Om vi queryar $v=1$ kan det endast finnas x_i till höger

Lösning:

1. Querya $v=1$
2. x_0 kommer finnas på $1+D_0$

Testfallsgrupp 2 (N=2)



Att querya $v=1$ kommer ge oss 2st D, vi vet inte ordningen

Vi kommer antingen få $x_0 = 1 + D_0$, eller $x_0 = 1 + D_1$

Insikt 2: Det finns en punkt på v om och endast om det finns en nolla i D (edge case)

Insikt 3: Om det inte finns en nolla i D för queryn $v=1$ måste alla $x_i > 1$

Insikt 3 innebär att exakt ett D_i kommer att minska med exakt 1 när vi går från $v=0$ till $v=1$

Lösning: querya $v=1$ och $v=2$, hitta det D_i som "byts ut". Detta D_i är distansen $v=1$ till x_0 , och det andra D_i är distansen från x_0 till x_1

Testfallsgrupp 3 ($x_i \leq N+1$)

Det finns x_i utplacerade på N utav $N + 1$ möjliga punkter, det finns en "hålrum"

Lösning: Querya $v=i$ för alla i där $1 \leq i < N + 1$. Om resultatet av v_i inte innehåller en nolla finns "hålrummet" på plats i . Om detta inte händer finns "hålrummet" på plats $N+1$

Testfallsgrupp 4 ($N \leq 100$, $x_i \leq 10^4$, $N + 1$ queries)

Insikt 4: om vi vet vad distansen mellan alla plankor är, och vi vet ett x_i , kan vi hitta $x_{(i+1)}$ genom att querya $v = x_i + 1$, och hitta den distansen D_k som har "försvunnit". Då är $x_{(i+1)} = x_i + D_k$

Lösning:

1. Hitta x_0 på samma sätt som för testfallsgrupp 2 (2 queries)
2. Vi kan hitta alla distanser mellan de olika x_i m.h.a de 2 första queries
3. Därefter kan vi använda Insikt 4 för att hitta resten av $x:n$ (1 query per x , $N-1$ queries)

Allmän lösning

Insikt 5: genom att querya $v=1$ får vi ut att x_n = summan av alla D_i

Detta sparar en query från testfall 4

Det finns lite specialfall man måste akta sig för

Hoppers

Av: Joshua Andersson

Hoppers

Problem: Givet en speciell riktad graf med saker i dess noder, hitta när kollisioner sker.

Grupp	Poäng	Gränser
1	15	$X \cdot Y \leq 1000, N \leq 1000$
2	25	$N \leq 1000$
3	10	Om en hopper inte är en del av en cykel kan max en annan hopper peka in i den.
4	35	Det finns inga hoppers som pekar uppåt.
5	15	Inga ytterligare begränsningar.

Grupp 1 ($R * C \leq 1000, N \leq 1000$)

- Liten indata. Brute force
- Varje sak kan max röra sig $R \cdot C$ steg innan den går in i en cykel
- Flytta alla framåt och se om kollisioner sker. $O(N)$
- Totalt $O(RCN)$ tid

Grupp 2 ($N \leq 1000$)

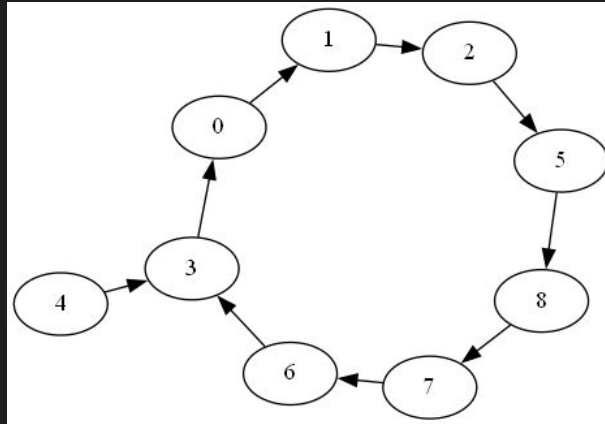
- Insikt: väldigt särskild sorts graf
- Cykler

>>V

^<V

^<<

=



Grupp 2 ($N \leq 1000$)

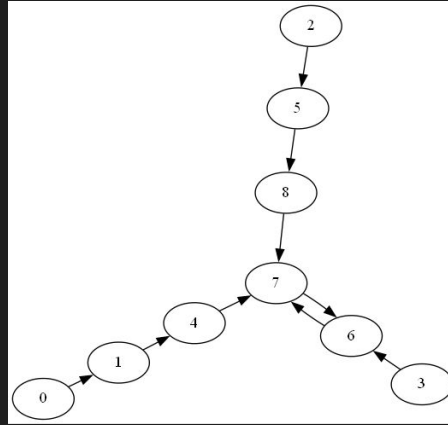
- **Insikt: väldigt särskild sorts graf**
- **Träd**

>vv

vvv

><<

=



Grupp 2 ($N \leq 1000$)

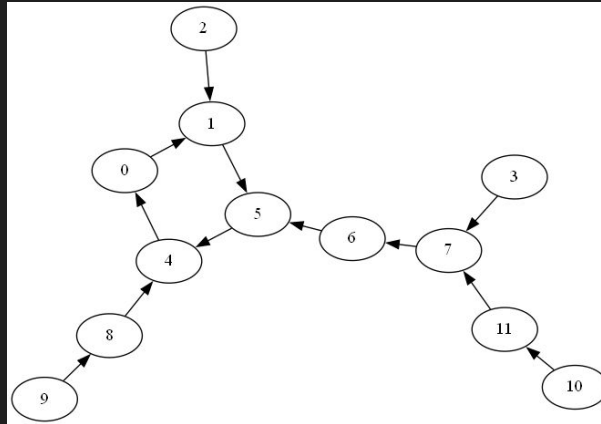
- **Insikt: väldigt särskild sorts graf**
- **Funktionell graf (kan beskrivas av en funktion: $f(\text{nod}) = \text{nästa nod}$)**
- **Varje komponent är en cykel med träd som pekar in på noder i cykeln**

>v<v

^<<<

^<>^

=



Grupp 2 ($N \leq 1000$)

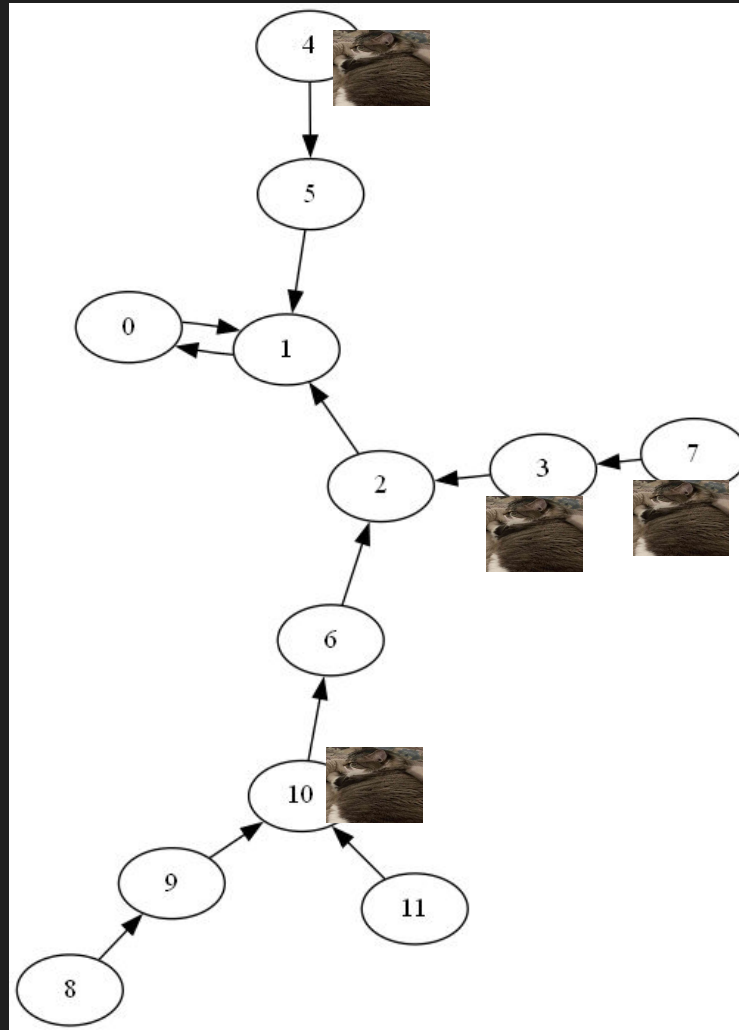
- Vi delar in i cykler/träd
- Cykler: för varje cykel kan vi generera en lista med par $[t, [x, y]]$: en sak anländer vid nod $[x, y]$ vid tid t . Sortera listan efter tid, flytta fram alla noder i cykeln $t - t_{\text{last}}$ steg i $O(n)$ tid. Kolla om $[x, y]$ krockar med något. Görs $O(n)$ gånger, $O(n^2)$
- (Representera positioner som index i cykeln. $\text{pos}[i] = (\text{pos}[i] + (t - t_{\text{last}})) \% \text{cycle_len}$)

Grupp 2 ($N \leq 1000$)

- Vi delar in i cykler/träd
- Träd:
- **Insikt: två noder kan bara krocka om de är av samma avstånd från roten.**
- **Gör en dfs. För varje nod, ha en dictionary/map över vilka items som finns djupare i trädet. Kombinera rekursivt med barnen. Om det någonsin finns två med samma tid sker en krock. (Pitfall: du kan inte ta bort den direkt, kan ske en tredje krock. Sätt istället värdet till inf+positionens index. Lite pill men inte för hemskt)**

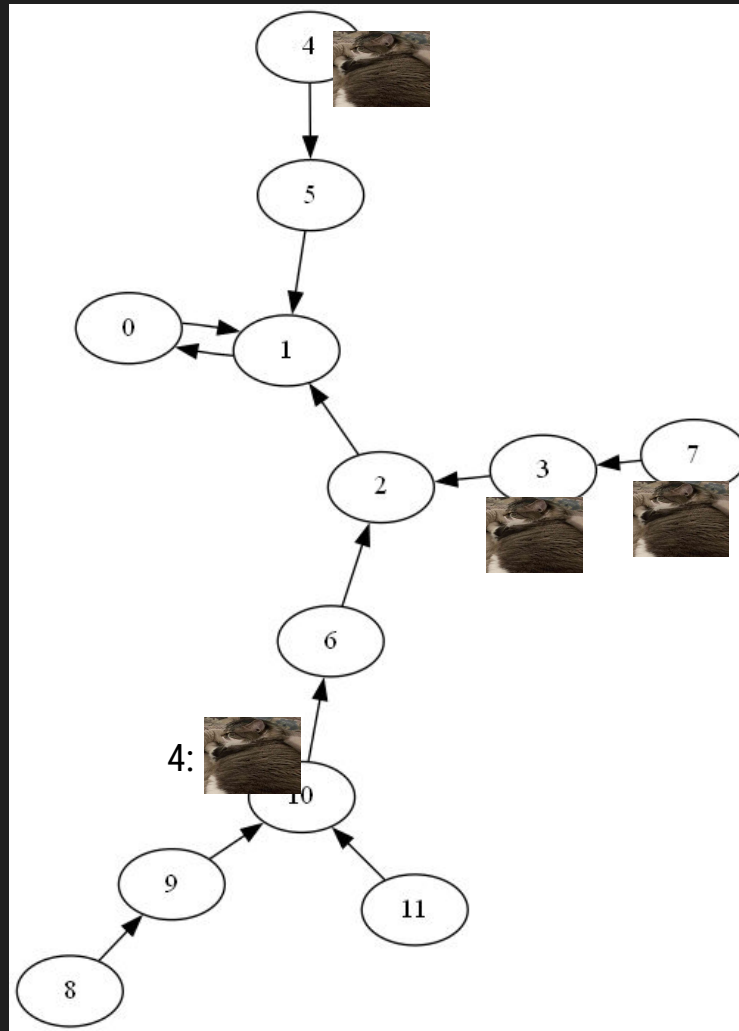
Grupp 2 ($N \leq 1000$)

- Vi delar in i cykler/träd
- Ursprungsstate



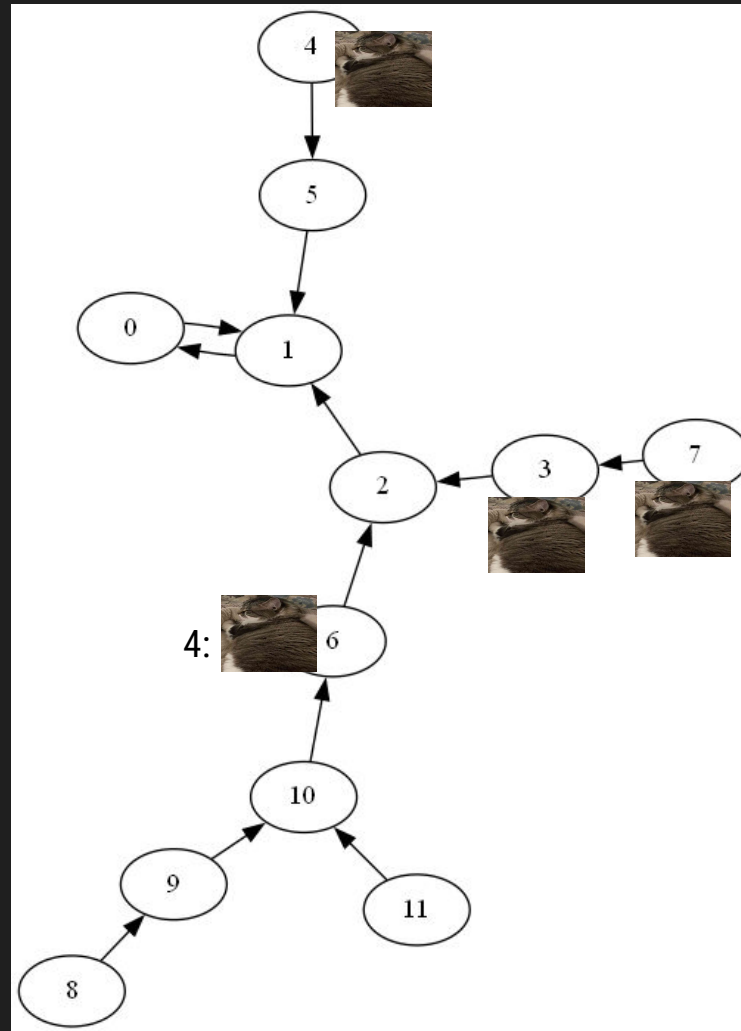
Grupp 2 ($N \leq 1000$)

- Vi delar in i cykler/träd
- Ursprungsstate



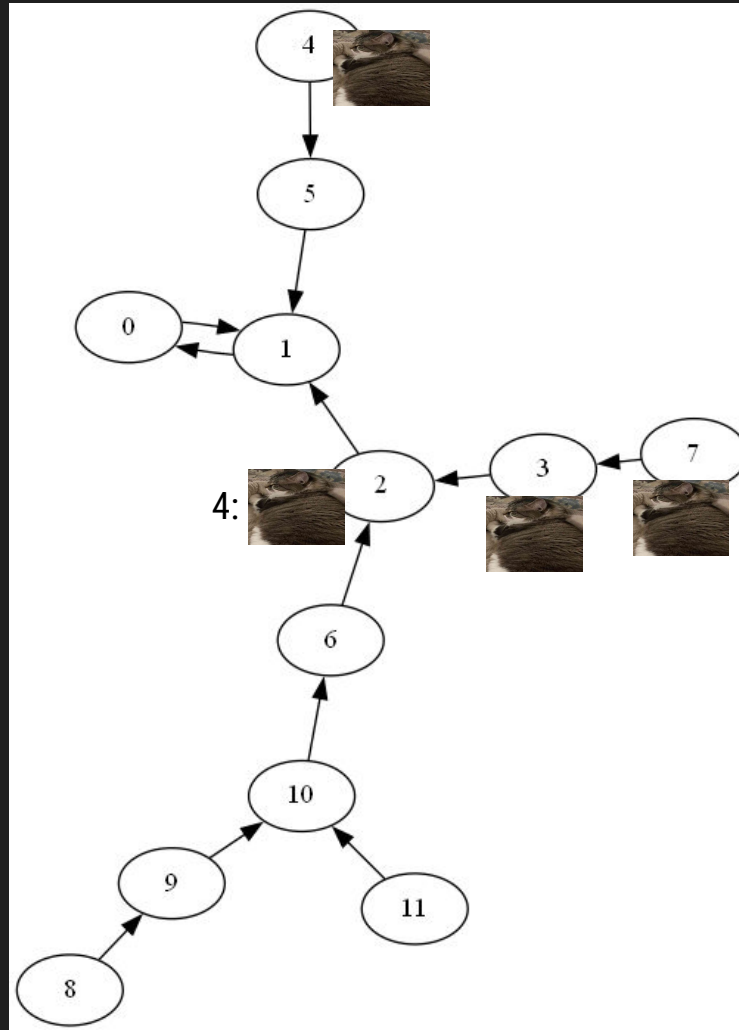
Grupp 2 ($N \leq 1000$)

- Vi delar in i cykler/träd
- Ursprungsstate



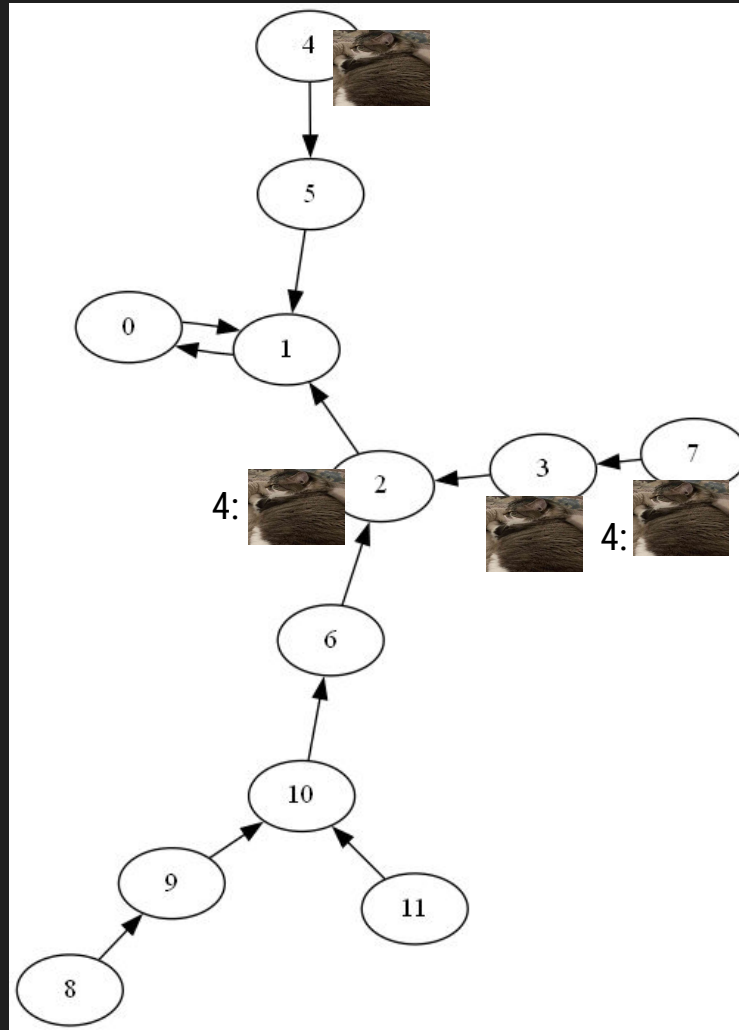
Grupp 2 ($N \leq 1000$)

- Vi delar in i cykler/träd
- Ursprungsstate



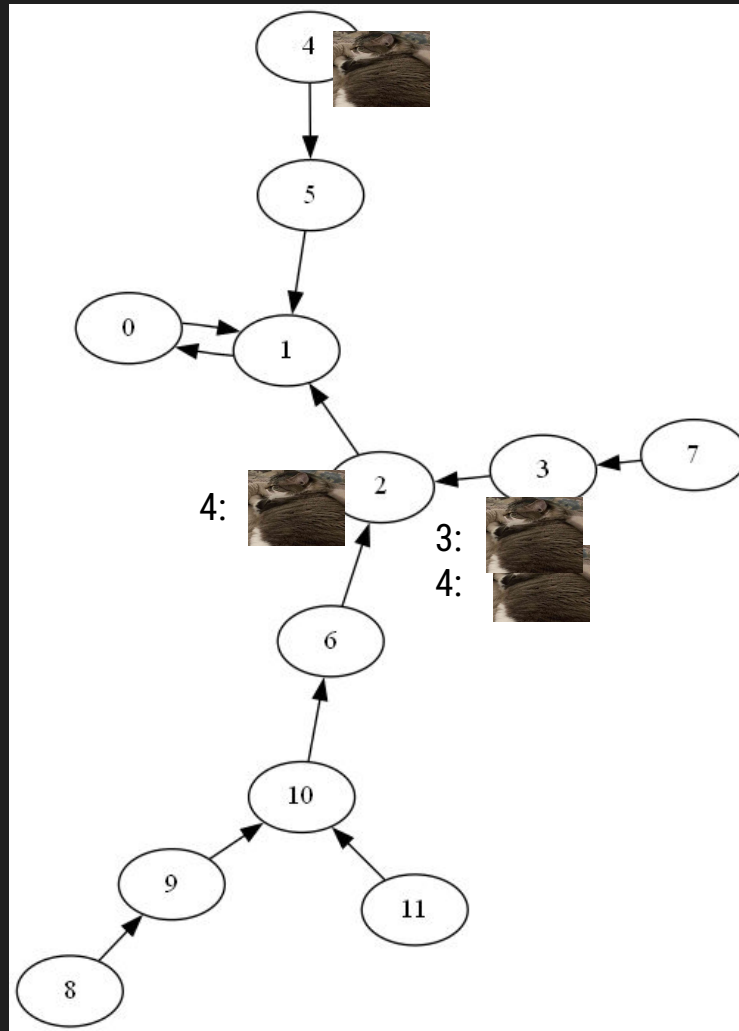
Grupp 2 ($N \leq 1000$)

- Vi delar in i cykler/träd
- Ursprungsstate



Grupp 2 ($N \leq 1000$)

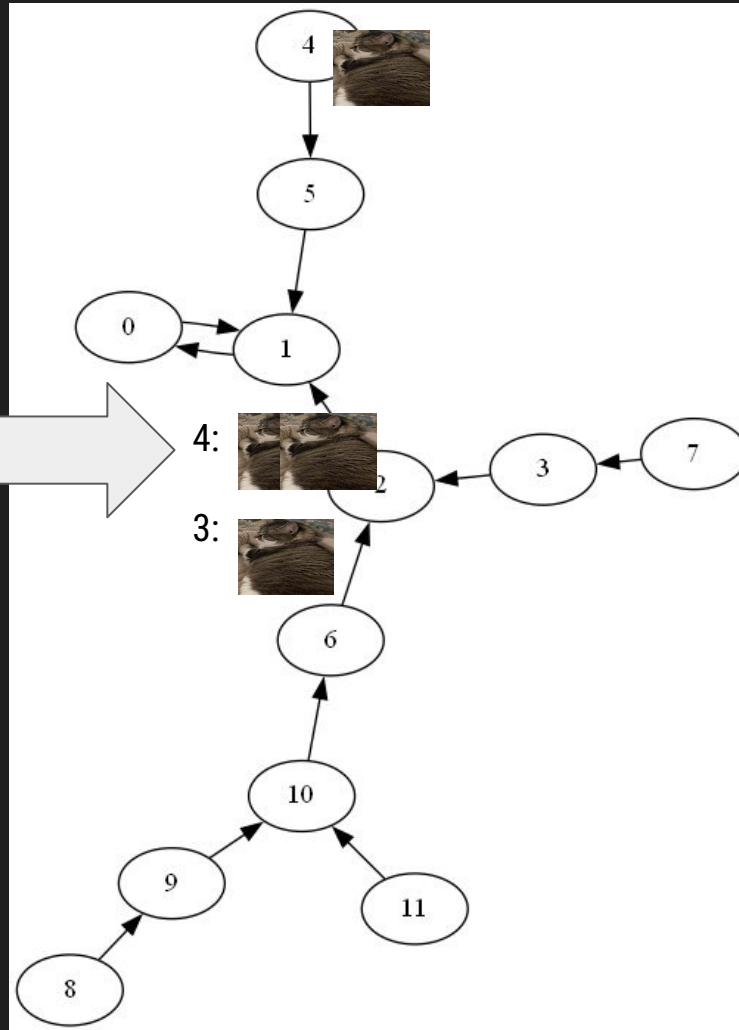
- Vi delar in i cykler/träd
- Ursprungsstate



Grupp 2 ($N \leq 1000$)

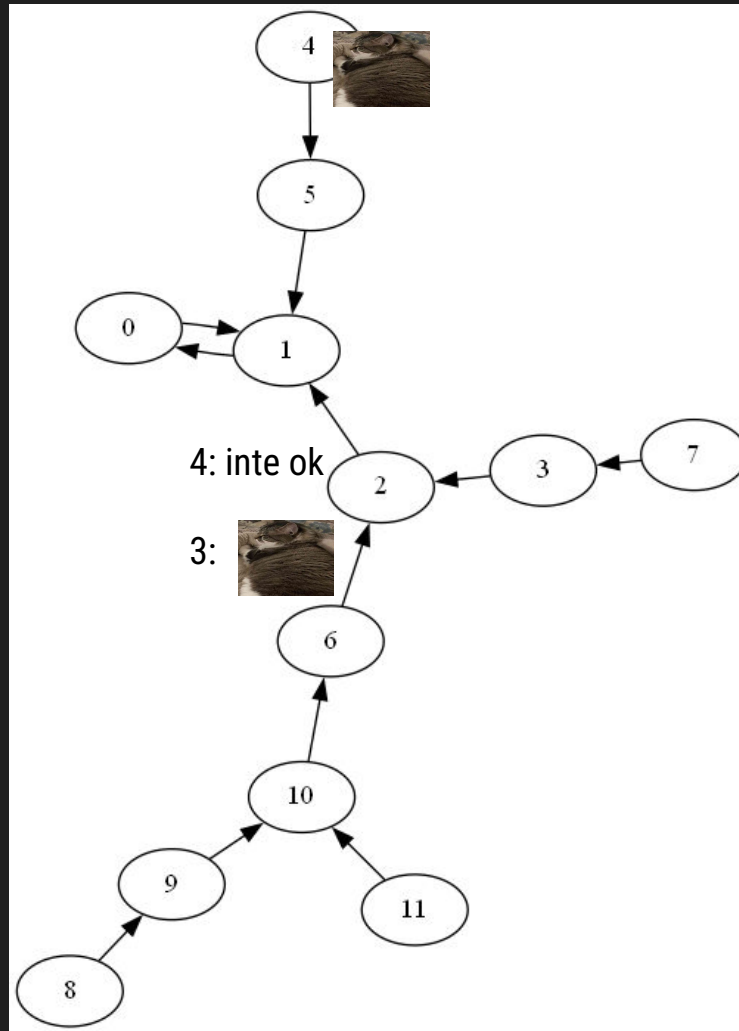
- Vi delar in i cykler/träd
- Ursprungsstate

Kollision!!



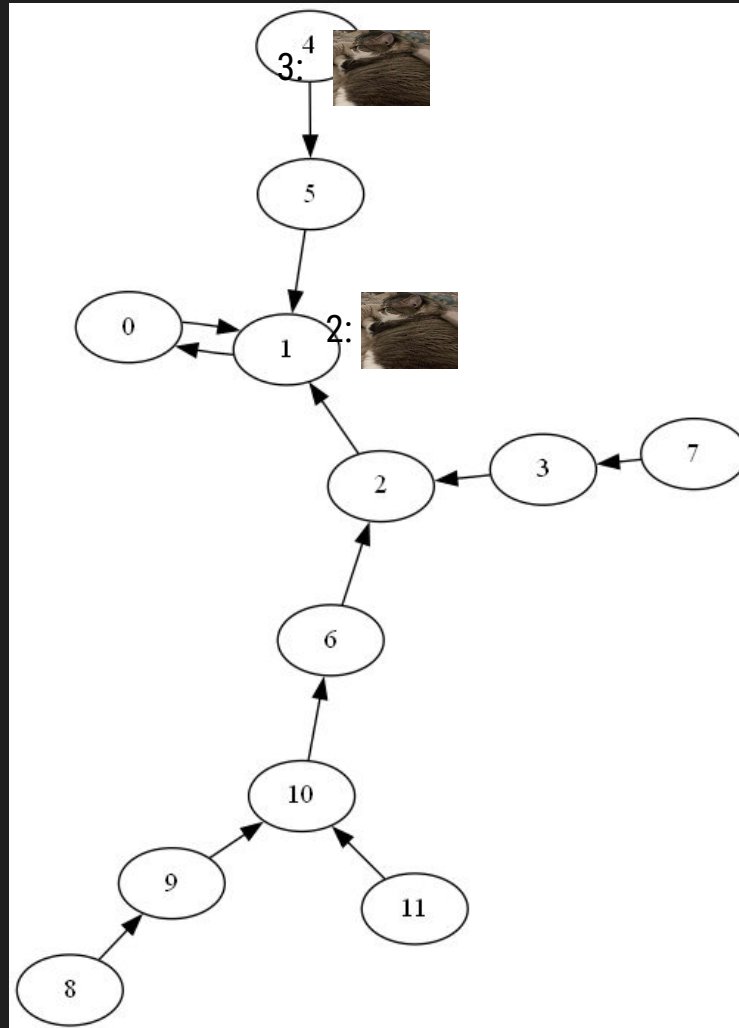
Grupp 2 ($N \leq 1000$)

- Vi delar in i cykler/träd
- Ursprungsstate



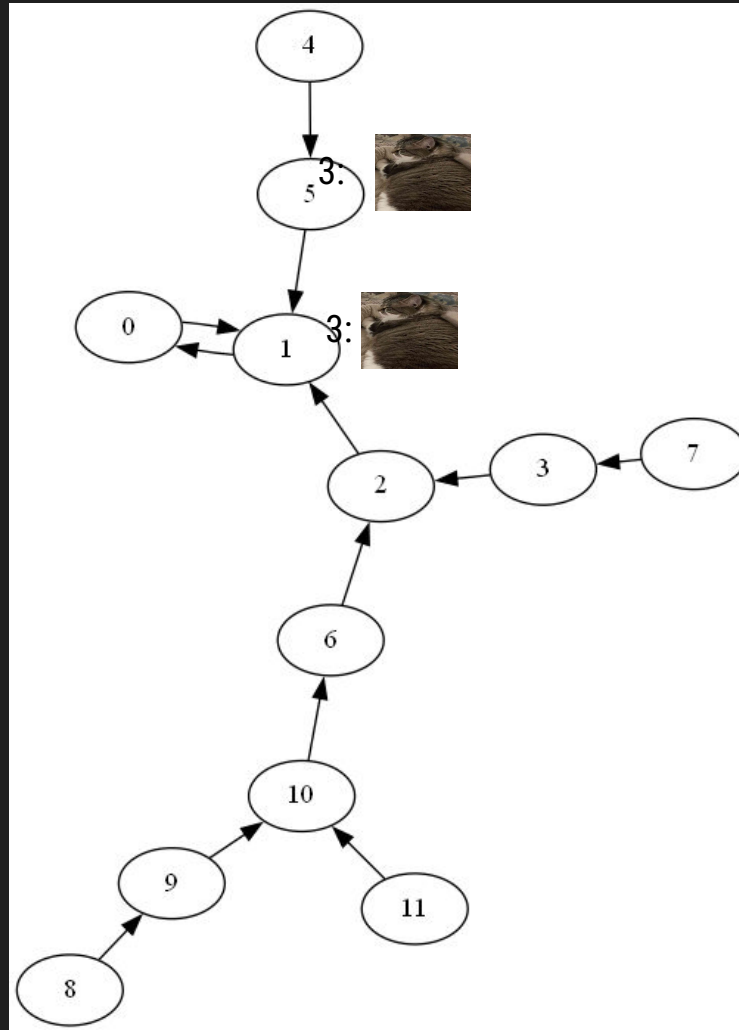
Grupp 2 ($N \leq 1000$)

- Vi delar in i cykler/träd
- Ursprungsstate



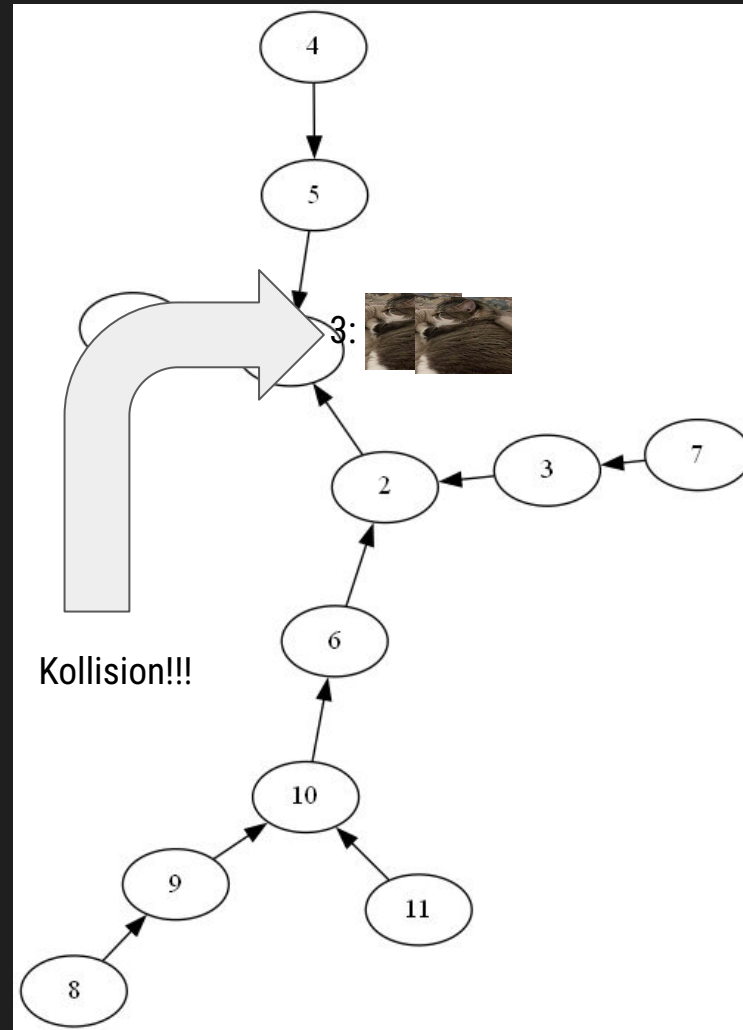
Grupp 2 ($N \leq 1000$)

- Vi delar in i cykler/träd
- Ursprungsstate



Grupp 2 ($N \leq 1000$)

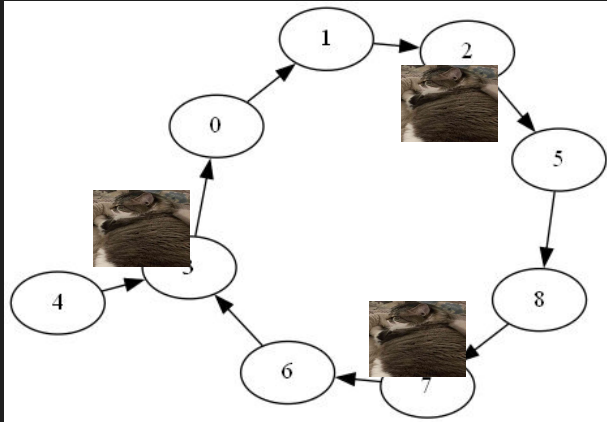
- Vi delar in i cykler/träd
- Ursprungsstate



Grupp 3 (Alla träd är linjer)

- Lite konstig formulering
- Våra träd är linjer => krockar kan bara ske inuti cykler. Hitta när sakerna går in i cykler i $O(n)$ (triviellt, det är bara hur djupt ner i trädet saken ligger)
- Hantera cykler i $O(1)$ per sak vi lägger in.
- Istället för att flytta alla noder i cykeln framåt, flytta innoden bakåt

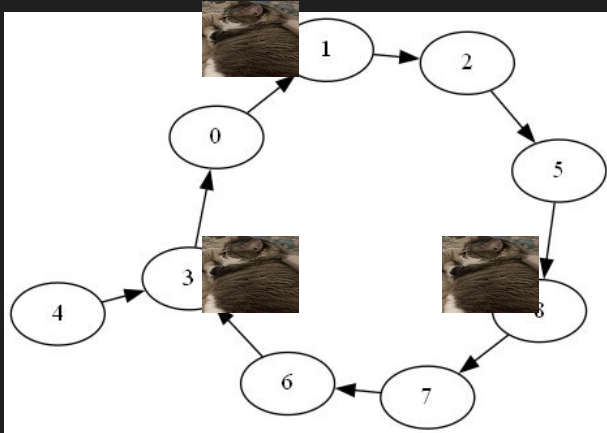
t=0



Grupp 3 (Alla träd är linjer)

- Lite konstig formulering
- Våra träd är linjer => krockar kan bara ske inuti cykler. Hitta när sakerna går in i cykler i $O(n)$ (triviellt, det är bara hur djupt ner i trädet saken ligger)
- Hantera cykler i $O(1)$ per sak vi lägger in.
- Istället för att flytta alla noder i cykeln framåt, flytta innoden bakåt

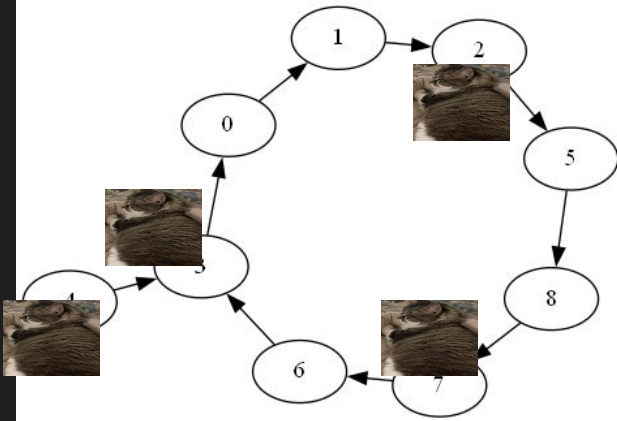
t=2



Grupp 3 (Alla träd är linjer)

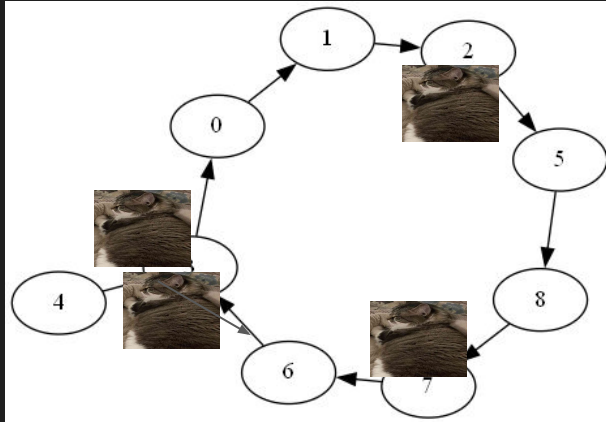
- Lite konstig formulering
- Våra träd är linjer => krockar kan bara ske inuti cykler. Hitta när sakerna går in i cykler i $O(n)$ (triviellt, det är bara hur djupt ner i trädet saken ligger)
- Hantera cykler i $O(1)$ per sak vi lägger in.
- Istället för att flytta alla noder i cykeln framåt, flytta innoden bakåt

t=0



Grupp 3 (Alla träd är linjer)

- Lite konstig formulering
- Våra träd är linjer => krockar kan bara ske inuti cykler. Hitta när sakerna går in i cykler i $O(n)$ (triviellt, det är bara hur djupt ner i trädet saken ligger)
- Hantera cykler i $O(1)$ per sak vi lägger in.
- Istället för att flytta alla noder i cykeln framåt, flytta innoden bakåt



anländer $t=1$
flytta bak 1 steg,
finns nåt där?

Grupp 4 (Inga uppåtpilar)

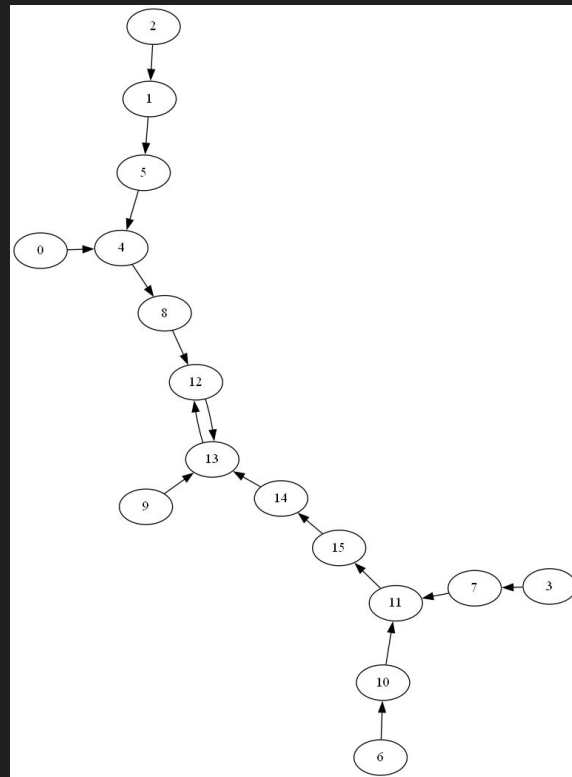
- Cykler kan max ha längd 2
- Svagare testdata
- Stora träd

VV<V

V<VV

VV>V

><<<



Grupp 4 (Inga uppåtpilar)

- Cykler kan max ha längd 2
- All tid spenderas att flytta noder från ett set till ett annat. Kan vi göra bättre? JA!

Smaller to larger merging. Vi vill merge a och b:

```
if (a.size() < b.size()) swap(a, b);  
for (auto& item : b) a.insert(item);
```

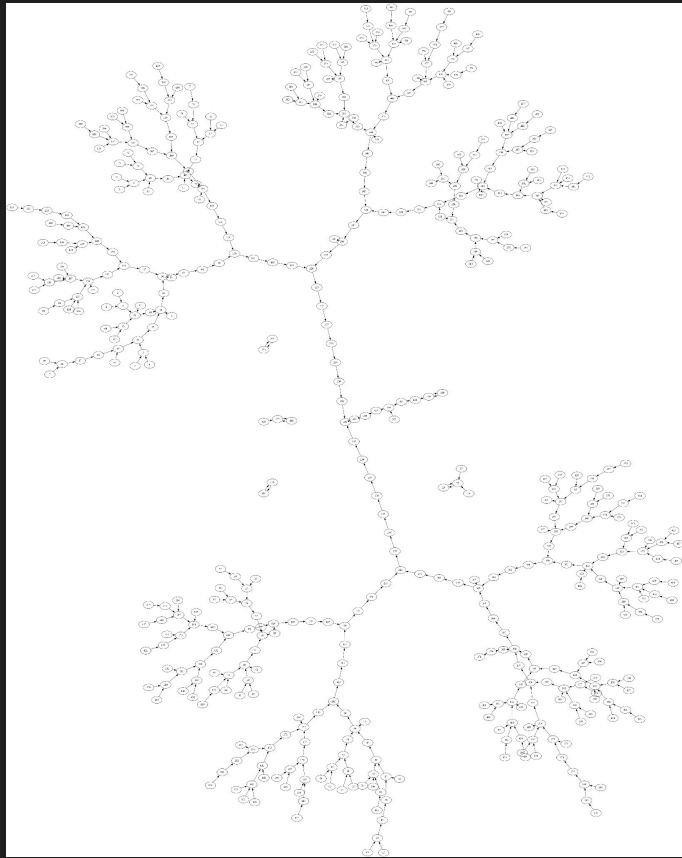
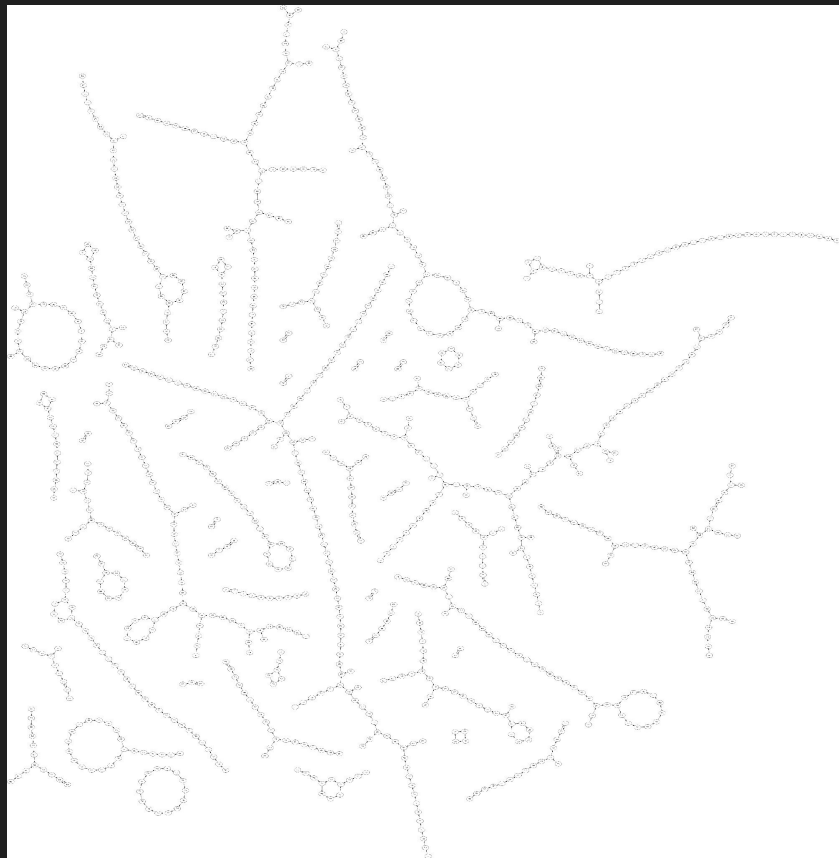
Ett givet element kan max byta sen $\log(n)$ gånger. Först i ett set av storlek 1,2,4,8,...

- Förvånansvärt svårt att implementera buggfritt (blir lite komplicerat när man skickar vidare "kollision skedde här", inte svårt men lätt att få småfel (domare gör aldrig fel lololol))

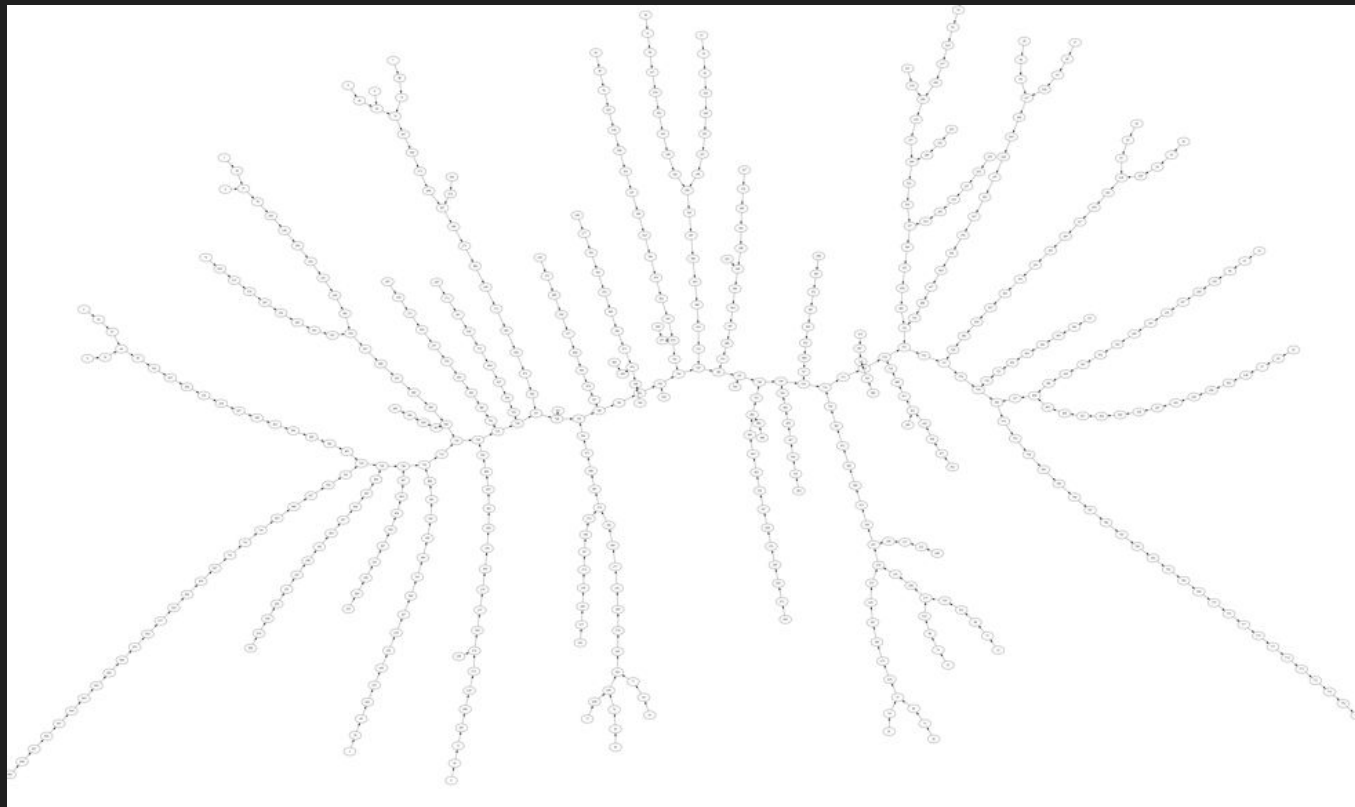
Grupp 5 (Full)

- **Kombinera lösningen för grupp 3 och grupp 4!**

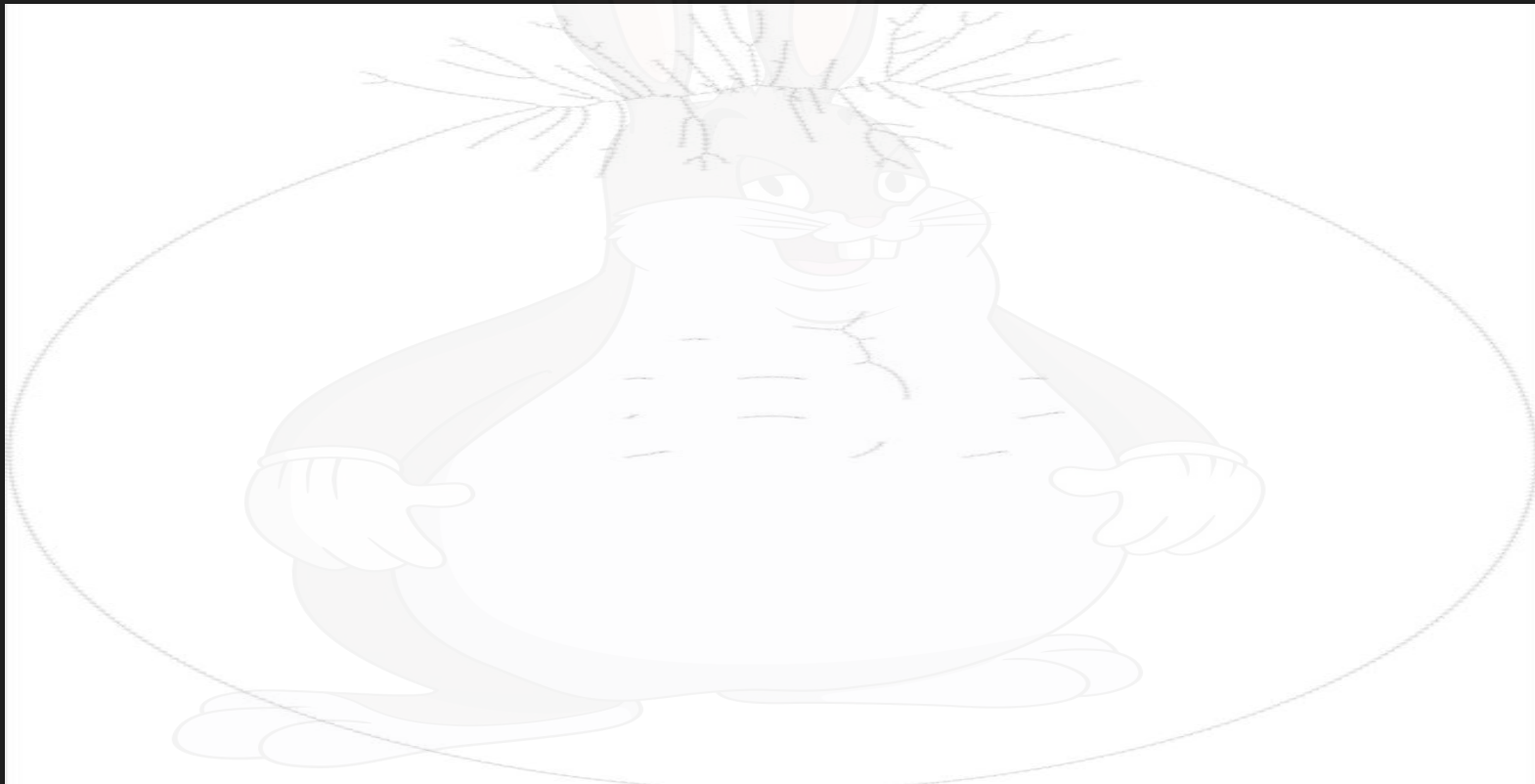
Roliga grafer!



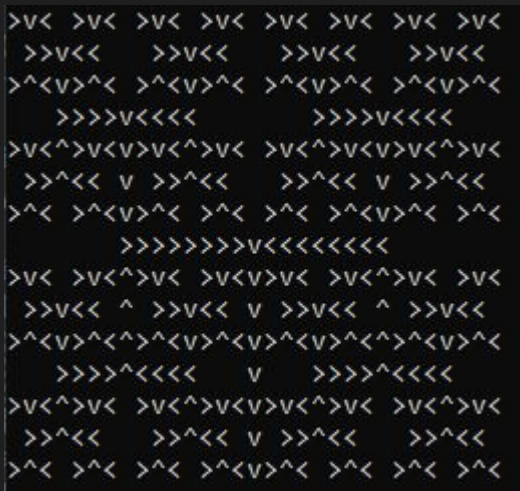
Roliga grafer!



Roliga grafer!



Fraktaler (tack Simon)

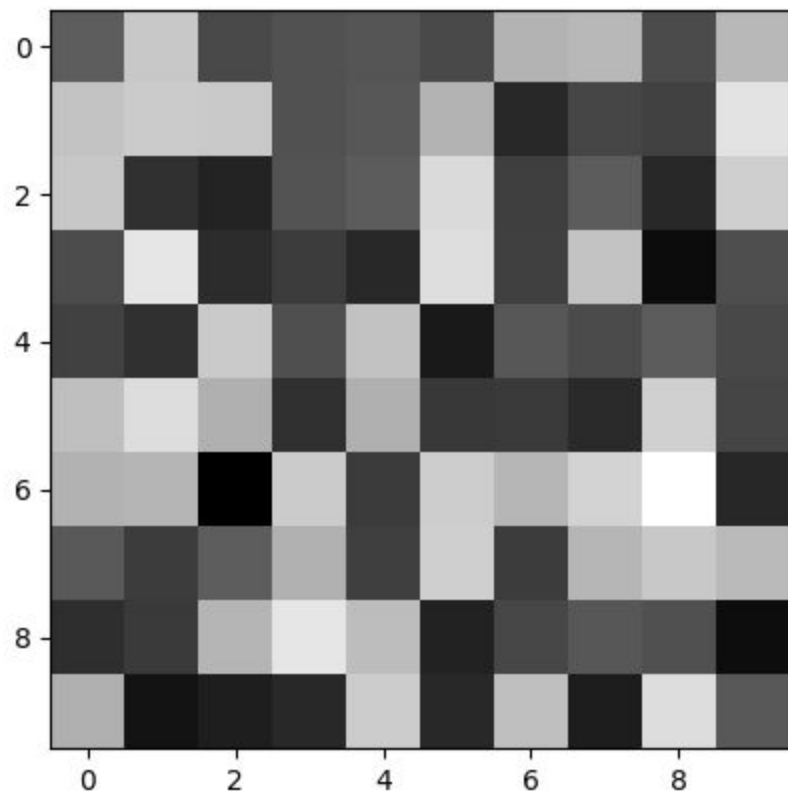


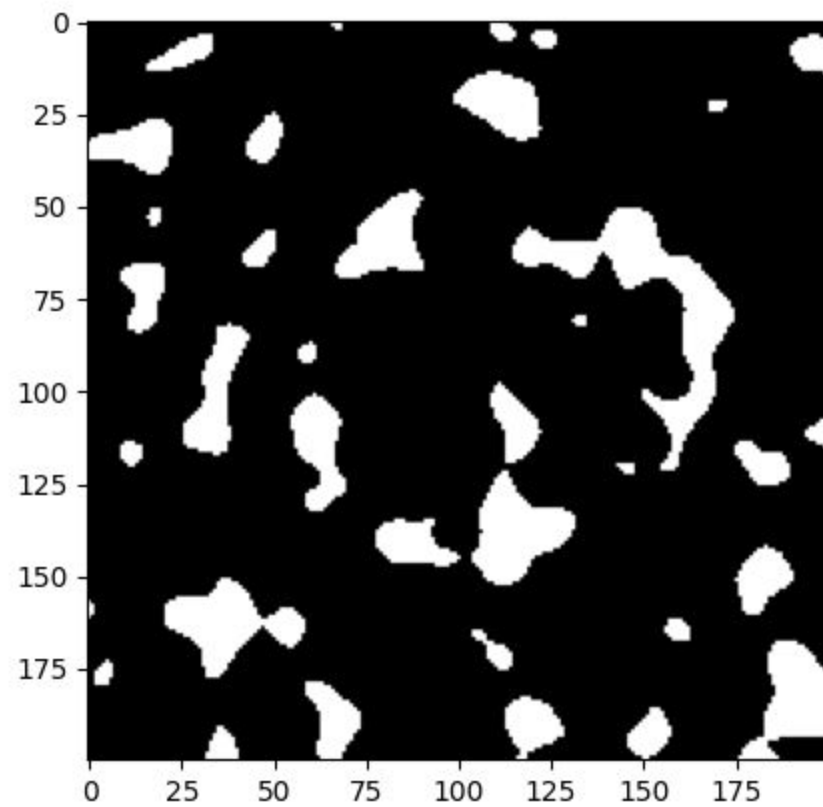
32 63 0

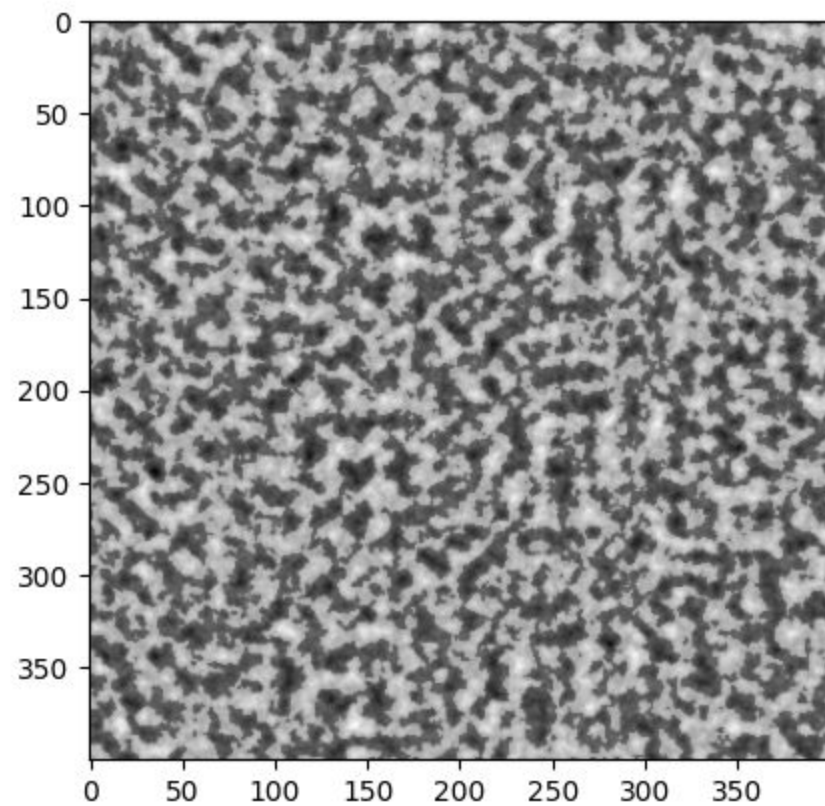
[illegible]

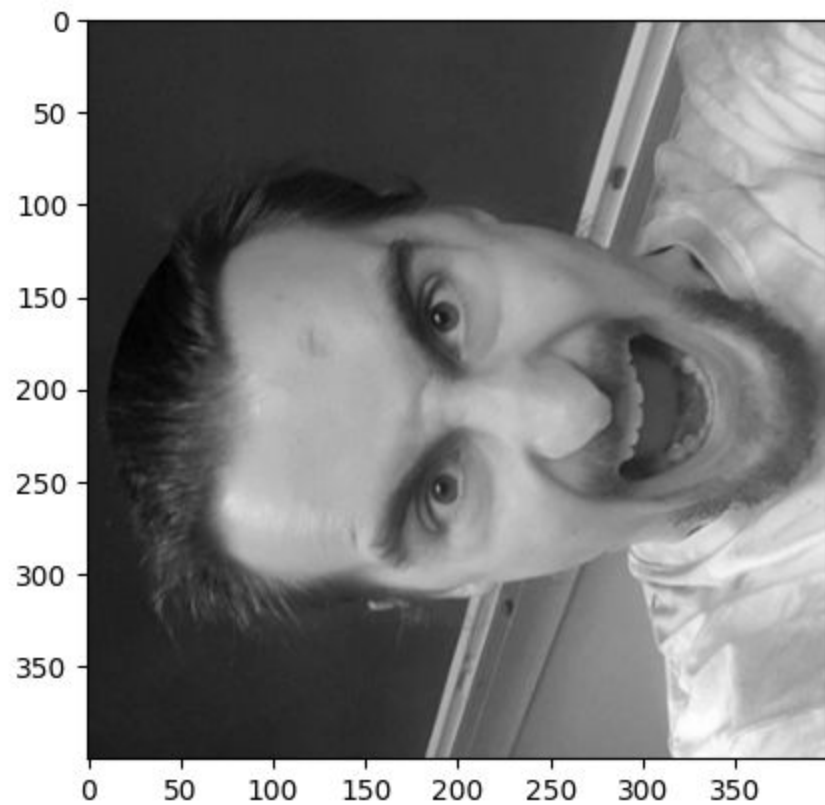
Berguppdelning

Av: Olle Lapidus, Alexander Wahlsten









IOI-uttagningen

Internationella mästerskap

- IOI
 - Top 4 i uttagningen
 - Oklart datum, oklart online/onsite/baltisk samling
- BOI
 - Top 4 i uttagning + 2 till (oftast, men inte alltid, 2 bästa icke-treorna)
 - Oklart datum, kanske senarelagt och onsite

Uttagningsmoment

- Finalen (100)
- Lägret (100)
 - 5 februari (5h på morgonen)
- KATT (100)
 - v7-v9 (under valfria 24h)
- Nordiska (100)
 - ???
- Teori 1 (50)
 - Fram till ???
- Teori 2 (50)
 - ??? till ???
- Alla moment summeras för uttagningen - poängen skalas om (så 600p i finalen = 100p)

Teoriblad

- 4 svåra problem som löses på papper. Inget kodande krävs, endast pseudokod.
- Detaljer kring bedömning etc står på teoribladet
- Inlämning via mail

Upsolving

- Samtliga kodtävlingar kan **fram tills nästa tävling** *upsolvas*
- Ni kan fortsätta skicka in lösningar på kattis
- Om ni efter 1 vecka har X poäng och vid tävlingsslut har Y poäng får ni $(X - Y) / 4$ extrapoäng
- Dvs: nollade ni ett problem kan ni ändå höja er totalpoäng med 25 för uttagningen!
- Varje tävling har lösningsgenomgång, och ni får diskutera hur man löser problemen med varandra (och be om hjälp på discord), men ni måste *koda själva* - d.v.s. inte hjälpa varandra med själva implementeringen

Andra tävlingar

- Codingcup: En serie tävlingar som vanligtvis ordnas på olika ställen i Sverige. PO-finalen är en av deltävlingarna. <https://codingcup.se/>
- ICPC: Den största universitetstävlingen, där man deltar i lag med 1-3 personer. Drar igång med NCPC på hösten.
- Code Jam: Googles årliga tävling, brukar vara på våren.
- Hackercup: Facebooks motsvarighet.

Onlinedomare

- codeforces.com: Ordnar tävlingar ganska regelbundet (~en gång i veckan). Har även ett forum.
- atcoder.jp: Har också tävlingar ungefär en gång i veckan. Har ofta mer matematiska problem.

Kodsport

- <http://www.kodsport.se/> -> Bli medlem!
- Helt gratis att gå med
- Nyhetsbrev om våra aktiviteter
- Mer bidragspengar från vårt moderförbund, Unga Forskare